第 2 章練習解答　作業系統結構

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS02-E01　API 與 system call

1. 核心模式至少包含 system call 入口、參數驗證、目標物件權限檢查、受保護佇列操作及必要的排程喚醒。程式庫的格式整理可留在使用者模式。
2. 不一定。成功可能只表示資料已複製到核心或裝置佇列。持久性需要介面明確承諾 write-through、flush 或事務提交，並由儲存裝置確認。
3. 需驗證指標範圍、可讀性、長度乘法溢位、物件是否仍在生命期內；複製後再使用核心自己的緩衝區，避免使用者事後修改。

檢查。 三問分別對應模式、持久性與邊界驗證。

OS02-E02　Microkernel 取捨

1. 服務在各自 domain 執行，單一服務壞掉時未必能直接改寫核心記憶體；權限較小，重啟局部服務也較容易。
2. 跨 domain IPC 增加上下文切換與訊息複製成本；服務管理器、IPC 與 capability 傳遞若設計錯誤會形成新的高權限 attack surface；效能壓力也可能迫使過度共享。
3. 以 capability 最小化每個服務可存取的資源，並對 IPC parser、權限降級與崩潰恢復做模糊測試和端到端驗證。

檢查。 答案必須同時解釋 isolation 與 IPC boundary。

OS02-E03　開機信任鏈

1. 不能。若 boot loader 已不受信任，它可在載入前修改核心、改寫驗證結果或傳遞錯誤設定。
2. 前一階段需驗證下一階段映像，從不可變信任根逐步延伸。驗證結果也必須由後續階段實際依據，不能只記錄 log。
3. 需限制可接受簽章者、檢查撤銷狀態、版本與政策，並避免執行時任意降低簽章驗證。簽章只證來源與完整性，不證明程式碼無漏洞。

檢查。 信任鏈必須每段都成立，任一交接點失守即失效。

第 2 章練習解答　作業系統結構 （續）

OS02-E04　Short write與複製不變量

1. read4096後來源4096、目的0；write1000後目的1000、剩3096；write3096後目的4096；read1904後來源6000；write1024後目的5120、剩880；write880後目的6000；read0後EOF。read正bytes總4096+1904=6000，write總1000+3096+1024+880=6000。
2. 第一段只寫1000，丟掉3096；第二段只寫1024，丟掉880；總漏3976，輸出僅2024。後段不能因讀到EOF而補回已覆寫buffer。
3. 已寫W≤已讀R；buffer[j:n]恰是尚未寫出的來源連續片段。正回傳k使j與W同增k，不重複已完成前綴；j=n才讀下一段。零進度EINTR重試同一剩餘區間；正回傳先推進。write0需回報/避免無限迴圈。EOF且buffer清空才能推出W=R。
4. 不是。file fsync在契約與device flush成立下同步file data/必要metadata；directory entry另需directory fsync。這個copy直接暴露部分目的，不具atomic replace；若需求替換舊檔需另設暫存檔/同步/rename/directory同步協議。

檢查。 數值、待寫前綴與持久化對象都要逐一對上。

OS02-E05　Pipeline端點與EOF

1. cat保留fd1→write end，關fd3與原fd4；wc保留fd0→read end，關原fd3與fd4；shell關fd3/fd4。原fd與dup2後的引用都算，不能只看descriptor編號是否與原pipe相同。
2. shell或wc仍持有write-end descriptor。即使資料已讀空，只要writer引用存在，blocking read仍可等待未來資料；shell若同時wait wc會陷入不必要僵持。
3. 不矛盾。cat未寫bytes並關閉write end；wc見空stream EOF，正確計數0並成功結束。pipeline整體exit規則不等於每個stage成功，需取得兩個wait statuses。
4. 不會。本模型fork建立兩個children，exec各自替換其image而維持process身分；shell未被exec且仍等待。

檢查。 以引用而非程式名稱判EOF。

第 2 章練習解答　作業系統結構 （續）

OS02-E06　ABI範圍與TOCTOU

1. 24+8=32bytes，僅此明定的memory-block ABI計數。API給行為/型別，ABI給machine layout/entry/return，implementation做path/permission/fd處理；不能推Linux真open一律copy32bytes。
2. p<U且n≤U−p。U−p=0x100，n=0x100合法，exclusive end C0000000而最後byte BFFFFFFF；n=0x101跨界。mapping/permission與實際安全copy仍另驗。
3. 數學值100000004(hex)，32-bit只剩4。先檢count≤SIZE_MAX/4=3FFFFFFF，故拒絕；不能先wrap再配置4bytes並照原count寫。
4. 另一thread可改mapping，真正存取可fault，check與use之間也能改user參數。需fault-aware user copy、kernel-owned bounded副本與內部格式驗證；namespace穩定性還需核心object references/同步。

檢查。 數值範圍合法不是存取必然成功。

OS02-E07　Policy切換與layer依賴

1. q1區間A0–1 B1–2 C2–3 A3–4 B4–5 A5–6 B6–7 A7–8 A8–9；completion9,7,3，response0,1,2。q4 A0–4 B4–7 C7–8 A8–9；completion9,7,8，response0,4,7。
2. q1平均1ms、q4平均11/3ms；相鄰不同task共7vs3次，最後A→A不算task switch。非零switch成本時wall times/response需再計，不能沿用零成本表。
3. timer、context保存/恢復、受控ready queue是mechanisms；選RR、q=1或4與queue順序是policy。硬體timer精度與nonpreemptible長度仍限制可達性。
4. 每條依賴都要求callee layer更低，沿環會得到l(pager)>l(driver)>l(scheduler)>l(pager)，矛盾。把scheduler必要metadata與前進所需核心資源保留resident，底層scheduler primitive不再依賴pager；其他工作再依契約分層。

檢查。 同時給實際排程和一般循環矛盾。

第 2 章練習解答　作業系統結構 （續）

OS02-E08　Trace、尾端延遲與race反例

1. CPU[0,1)+[9,10)=2；blocked[1,8)=7；ready[8,9)=1；總10。裝置完成t8不等於task回傳或立即running。
2. 總107，mean13.375ms；ceil(.99×8)=8，p99取第8筆=100ms。樣本很小，該order statistic不等於已估穩定母體p99。
3. T1讀0、T2讀0、T2寫1、T1寫1，結果1；printf若改變排程成T1讀寫完再T2讀寫則結果2。觀測改時序而不是同步；不再重現不足以排除data race。
4. 不可，兩者仍可在加鎖前讀0，然後序列寫1。需同一mutex保護完整read-modify-write或適合的atomic RMW；用不變量排除所有容許交錯，而非只檢已曝光trace。

檢查。 不把CPU sampling與blocked latency混為一談。

OS02-E09　Root依賴閉包與信任鏈

1. 不能。讀取S module先需root可讀，root可讀又需S driver，形成依賴環。把S built-in或放到可讀initramfs且依賴齊才排除該缺件。
2. 閉包只證必需元件可依序取得，尚需ABI相容、hardware正常、driver邏輯正確、root識別與init/loader內容正確。缺任一可在後續階段失敗。
3. 不夠。loader可跳過驗證、改結果或改image，可信前階段必須授權/驗證loader並強制policy；kernel/initramfs/config哪些被覆蓋要明定。
4. 測量記錄實際內容但未必阻擋；簽章驗證來源授權與完整性但不證無bug；anti-rollback/撤銷政策限制仍合法簽章的舊版本。三者不同，也不等於CPU/driver ABI相容。

檢查。 閉包、行為正確、來源可信三個命題分別證明。

第 2 章練習解答　作業系統結構 （續）

原創驗算程式附錄: system_boundary_models.py

```
"""Chapter 2: self-contained teaching models; not an OS/kernel emulator."""
from collections import deque


def copy_chunks(data, read_sizes, write_limits):
    """Preserve order under short reads/writes; each limit must make progress."""
    destination = bytearray()
    source_offset = 0
    writes = iter(write_limits)
    trace = []
    for requested in read_sizes:
        if requested <= 0:
            raise ValueError("positive read request required")
        chunk = data[source_offset:source_offset + requested]
        source_offset += len(chunk)
        trace.append(("read", len(chunk), source_offset, len(destination)))
        if not chunk:
            break
        consumed = 0
        while consumed < len(chunk):
            limit = next(writes)
            if limit <= 0:
                raise ValueError("write made no progress")
            count = min(limit, len(chunk) - consumed)
            destination.extend(chunk[consumed:consumed + count])
            consumed += count
            trace.append(("write", count, source_offset, len(destination)))
    return bytes(destination), trace


def user_range(pointer, length, upper=0xC0000000):
    """Only nonempty numeric ranges; mappings/faults are separate concerns."""
    return 0 <= pointer < upper and 0 < length <= upper - pointer


def allocation_bytes(count, element_size, word_bits=32):
    maximum = (1 << word_bits) - 1
    if count < 0 or element_size <= 0 or count > maximum // element_size:
        raise ValueError("allocation size overflows")
    return count * element_size


def round_robin(bursts, quantum):
    """All arrive at zero; zero switch cost; order is dictionary insertion order."""
```

第 2 章練習解答　作業系統結構 （續）

```
    if quantum <= 0 or any(value <= 0 for value in bursts.values()):
        raise ValueError("positive bursts/quantum required")
    remaining = dict(bursts)
    queue = deque(bursts)
    time = 0
    trace, first, completed = [], {}, {}
    while queue:
        name = queue.popleft()
        first.setdefault(name, time)
        duration = min(quantum, remaining[name])
        trace.append((time, time + duration, name))
        remaining[name] -= duration
        time += duration
        if remaining[name]:
            queue.append(name)
        else:
            completed[name] = time
    return {"trace": trace, "response": first, "completion": completed,
            "switches": sum(a[2] != b[2] for a, b in zip(trace, trace[1:]))}


def dependency_closure(available, requirements):
    available = set(available)
    while True:
        added = {name for name, needs in requirements.items()
                 if set(needs) <= available} - available
        if not added:
            return available
        available.update(added)


if __name__ == "__main__":
    source = bytes(index % 251 for index in range(6000))
    result, trace = copy_chunks(source, [4096, 4096, 4096], [1000, 3096, 1024, 880])
    assert result == source
    print("copy:", trace)
    for quantum in (1, 4):
        print("RR", quantum, round_robin({"A": 5, "B": 3, "C": 1}, quantum))
    print("numeric range:", user_range(0xBFFFFF00, 0x100), user_range(0xBFFFFF00, 0x101))
    dependencies = {"S": {"initramfs"}, "F": {"initramfs"}, "mount-root": {"S", "F"}}
    print("early dependencies:", sorted(dependency_closure({"K", "initramfs"},
dependencies)))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study